

KR24_SDC-II_III-I_2026 Project Review-I

Report on

**PERSONALIZED TECHNICAL INTERVIEW PREPARATION IN HIGHER
EDUCATION**

(SmartInterview — G-1392) | Date: 24-09-2026

Submitted in Partial fulfillment of requirements for B.Tech II Year II Semester course

By

1. SHAMAKURA SAITEJA GOUD (24BD1A665K)

2. THODSAM SRUJAN (24BD1A665R)

3. VASIREDDY VIGNESH REDDY (24BD1A665X)

4. AELUGU RANITH KUMAR (25BD5A6615)

5. KONDURI ABHIRAM (25BD5A6620)

Under the guidance of

Dr. TVG SRIDEVI

Department of Computer Science and Engineering (AI&ML)

Keshav Memorial Institute of Technology



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH.

Narayanaguda, Hyderabad, Telangana-29

2025-26



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH
Narayanaguda, Hyderabad, Telangana-29



CERTIFICATE

This is to certify that this is a bonafide record of the project report titled “Personalized Technical Interview Preparation in Higher Education (SmartInterview — G-1392)” for KR24_SDC-II_III-I_2026 Project Review-I which is being presented as the Real-time Research Project / Societal Related Project report by

1. SHAMAKURA SAITEJA GOUD (24BD1A665K)
2. THODSAM SRUJAN (24BD1A665R)
3. VASIREDDY VIGNESH REDDY (24BD1A665X)
4. AELUGU RANITH KUMAR (25BD5A6615)
5. KONDURI ABHIRAM (25BD5A6620)

In partial fulfillment for the II Year II Semester Course RTRP in KMIT affiliated to the Jawaharlal Nehru Technological University, Hyderabad

Faculty Mentor:

(Dr. TVG Sridevi)

Dept of CSE (AI&ML), KMIT

Program Coordinator

(Mr. Shailesh Gangakhedkar)

Submitted for Final Project Review held on: 24-09-2026

Vision& Mission of KMIT

Vision

- To be the fountain head in producing highly skilled, globally competent engineers.
- Producing quality graduates trained in the latest software technologies and related tools and striving to make India a world leader in software products and services.

Mission

- To provide a learning environment that inculcates problem solving skills, professional, ethical responsibilities, lifelong learning through multi model platforms and prepares students to become successful professionals.
- To establish an industry institute Interaction to make students ready for the industry.
- To provide exposure to students on the latest hard ware and software tools.
- To promote research-based projects/activities in the emerging areas of technology convergence.
- To encourage and enable students to not merely seek jobs from the industry but also to create new enterprises.
- To induce a spirit of nationalism which will enable the student to develop, understand India's challenges and to encourage them to develop effective solutions.
- To support the faculty to accelerate their learning curve to deliver excellent service to students.

PROGRAM OUTCOMES

PO1. Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions.

PO3. Design/Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs.

PO4. Conduct Investigations: Use research-based knowledge and methods including design of experiments, analysis and interpretation of data to provide valid conclusions.

PO5. Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools to complex engineering activities.

PO6. The Engineer and Society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues.

PO7. Environment and Sustainability: Understand the impact of professional engineering solutions in societal and environmental contexts.

PO8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. Individual and Teamwork: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large.

PO11. Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work.

PO12. Life-Long Learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning.



PROJECT OUTCOMES

P1: Implement deterministic PDF extraction for resumes and job descriptions with automated technical skill taxonomy mapping.

P2: Engineer a domain-filtered Retrieval-Augmented Generation (RAG) knowledge engine using ChromaDB and Sentence-BERT dense embeddings to ground interview questions without LLM hallucinations.

P3: Construct a deterministic adaptive state machine governed by Bloom's Revised Taxonomy to dynamically adjust question difficulty and cognitive depth based on candidate performance.

P4: Build a multi-factor answer evaluation engine combining LLM technical reasoning, semantic vector similarity, and concept coverage with real-time neural voice interaction.



MAPPING PROJECT OUTCOMES WITH PROGRAM OUTCOMES

PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
P1	H	H	M	H	H	L			M			M
P2	H	H	H	M	H							M
P3	M	H	H	H	H				M			H
P4	M	M	H	M	H				H	H	M	

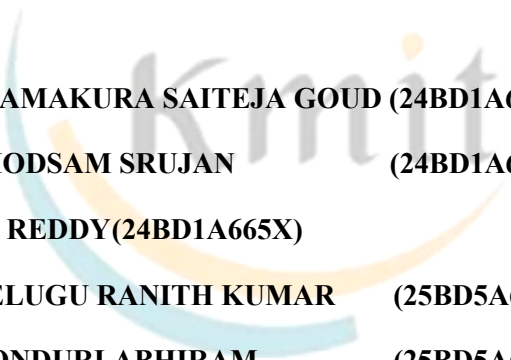


DECLARATION

We hereby declare that the results embodied in the dissertation entitled "Personalized Technical Interview Preparation in Higher Education (SmartInterview — G-1392)" for KR24_SDC-II_III-I_2026 Project Review-I has been carried out by us together during the academic year 2026 under the guidance of Dr. TVG Sridevi, Department of Computer Science and Engineering (AI&ML), Keshav Memorial Institute of Technology, and has not been submitted to any other University or Institute for the award of any degree.

Student Name

Roll No.

- 
- 1. SHAMAKURA SAITEJA GOUD (24BD1A665K)**
 - 2. THODSAM SRUJAN (24BD1A665R)**
 - 3. VASIREDDY VIGNESH REDDY(24BD1A665X)**
 - 4. AELUGU RANITH KUMAR (25BD5A6615)**
 - 5. KONDURI ABHIRAM (25BD5A6620)**

ACKNOWLEDGEMENT

We take this opportunity to thank all the people who have rendered their full support to our project work. We render our thanks to **Dr. B L Malleswari**, Principal, KMIT, for providing us all the required facilities.

We are grateful to **Mr. Neil Gogte, Founder & Director** and **Mr. S. Nitin, Director**, for facilitating all the amenities required for carrying out this project work.

We express our sincere gratitude to **Ms. Deepa Ganu, Academic Director** for providing an excellent environment in the college.

We are also thankful to mentors **Program Coordinator** for providing us with time to make this project a success within the given schedule.

We are deeply thankful to our Faculty Mentor Dr. TVG Sridevi, Department of Computer Science and Engineering (AI&ML), for her invaluable mentorship, continuous encouragement, and insightful feedback throughout the lifecycle of this project.

We would like to thank the entire KMIT faculty, who helped us directly and indirectly in the completion of the project.

We sincerely thank our friends and family for their constant motivation during the project work.



ABSTRACT

Technical recruitment interviews in software engineering demand multi-dimensional technical competency, clear conceptual articulation, and structured problem-solving under pressure. However, contemporary student preparation remains fundamentally bifurcated between static coding repositories (which fail to simulate interactive reasoning) and ungrounded commercial conversational AI chatbots (which suffer from hallucinations, lack pedagogical structure, and grade candidate responses subjectively). To resolve these systemic deficiencies, this project presents SmartInterview — a full-stack, decoupled, and pedagogically adaptive mock interview intelligence platform.

The architecture introduces three core technological pillars: (1) Domain-Filtered Retrieval-Augmented Generation (RAG) powered by ChromaDB and Sentence-BERT (all-MiniLM-L6-v2), indexing 402 curated computer science knowledge chunks across 8 foundational disciplines (DSA, System Design, DBMS, OS, Computer Networks, ML/DL, OOP, Design Patterns) to ensure zero question hallucination (Hit@1 = 96.97%, Hit@3 = 100%, MRR = 0.9798); (2) A Deterministic Adaptive State Machine governed by Bloom's Revised Taxonomy (Remember to Create), which dynamically raises or lowers cognitive complexity based on real-time performance thresholds (≥ 80 to advance, < 50 to remediate); and (3) An Objective Multi-Factor Scoring Engine evaluating answers across technical accuracy (30%), completeness (20%), relevance (20%), dense semantic cosine similarity (15%), and concept coverage (15%).

The platform incorporates real-time neural speech interaction utilizing Microsoft Edge-TTS and Groq Whisper STT, a high-availability dual-engine LLM client with sub-second latency failover (Groq Primary openai/gpt-oss-120b -> Groq Fallback openai/gpt-oss-20b -> Local Offline Ollama Llama 3.1), and automated ReportLab PDF diagnostic generation. Developed under the curriculum of Keshav Memorial Institute of Technology (KMIT), SmartInterview demonstrates production-grade full-stack engineering and reproducible AI alignment for higher education engineering career readiness.

LIST OF FIGURES

S.No	Name of Screenshot	Page No.
1.	SmartInterview 4-Tier System Architecture Diagram	24
2.	Dynamic Bloom's Taxonomy Pedagogical State Machine	30
3.	End-to-End Interview Execution Workflow Diagram	36
4.	Entity-Relationship Diagram (ERD) Schema	42
5.	Multi-Signal Answer Scoring & Evaluation Flowchart	48
6.	Comprehensive Candidate Feedback & PDF Analytics Dashboard	54



CONTENTS

CHAPTER 1 INTRODUCTION

1.1 Background and Motivation

In the contemporary technology industry, recruitment screening for software engineering, artificial intelligence, and systems infrastructure roles places heavy emphasis on technical interview performance. Candidates are expected not only to possess theoretical computer science knowledge, but also to articulate architectural trade-offs, explain algorithmic complexity, and structure technical reasoning interactively under scrutiny. However, traditional technical interview preparation in higher education faces significant structural bottlenecks.

Undergraduate engineering students traditionally rely on static problem repositories such as LeetCode, HackerRank, and GeeksforGeeks. While effective for syntax and algorithmic pattern matching, these platforms lack conversational interaction and cannot assess verbal articulation or contextual depth. Conversely, human-driven mock interviews (such as Pramp or campus placement drives) offer rich interactive feedback but suffer from severe scalability limits, scheduling friction, and high institutional costs. In response, many students have turned to generic commercial conversational AI chatbots (e.g., ChatGPT, Claude). However, unconstrained LLMs are plagued by three systemic flaws: (1) fact hallucination due to lack of domain-grounded retrieval, (2) arbitrary and subjective grading without mathematical consistency, and (3) an absence of pedagogical progression that adapts question difficulty to cognitive learning hierarchies.

To solve these systemic challenges, this project presents SmartInterview — a decoupled, full-stack, and pedagogically adaptive mock interview intelligence platform. SmartInterview grounds generative question generation in a curated vector knowledge base using Retrieval-Augmented Generation (RAG), drives cognitive progression through a deterministic state machine based on Bloom's Revised Taxonomy, and scores candidate responses using an objective multi-signal formula.

1.2 Problem Statement

How can a software engineering mock interview platform provide scalable, realistic, and objective technical interview practice that prevents generative AI hallucinations, enforces structured cognitive progression across academic taxonomies, and produces consistent multi-dimensional scoring without relying on costly proprietary cloud infrastructure or human interviewers?

SmartInterview resolves this problem by implementing a 4-tier decoupled architecture: (a) deterministic PDF resume and job description parsing via PyMuPDF with automated skill gap mapping, (b) verified domain vector retrieval via ChromaDB and Sentence-BERT (all-MiniLM-L6-v2), (c) a Python-based Bloom's Taxonomy state machine operating independently of LLM non-determinism, (d) a multi-signal scoring formula combining LLM critique with mathematical dense cosine similarity, and (e) high-availability dual-engine LLM inference (Groq Primary -> Groq Fallback -> Local Offline Ollama).

1.3 End Users

The SmartInterview system is designed to serve four primary categories of stakeholders:

- **Undergraduate & Graduate Engineering Students:** Candidates preparing for technical campus placements who require personalized, interactive, and role-aligned mock interviews.

- **University Placement & Training Cells (PAT):** Academic departments seeking to track cohort-wide technical competency, identify syllabus deficiencies, and conduct standardized mock assessments.
- **Corporate Recruiters & Talent Acquisition Teams:** Hiring organizations seeking automated, objective preliminary technical screening of high-volume applicant pools.
- **Self-Directed Software Engineering Professionals:** Working engineers transitioning to senior, full-stack, or machine learning roles who require rigorous system design and architectural practice.

1.4 Scope of the Project

The project scope encompasses the full-stack design, algorithmic implementation, empirical validation, and deployment of the SmartInterview platform. Key deliverables include: (1) an interactive React 18/Vite web application with dark-mode glassmorphic aesthetics and real-time audio playback; (2) a high-performance FastAPI asynchronous backend; (3) a curated 402-chunk computer science knowledge base indexed in ChromaDB across 8 core domains; (4) a 6-tier Bloom's Taxonomy state machine; (5) a 5-factor answer evaluation engine; (6) real-time neural speech synthesis (Edge-TTS) and transcription (Groq Whisper); and (7) dynamic ReportLab PDF report generation. Future enhancements such as multimodal video emotion analysis and multi-candidate panel simulations are formally documented as future scope.

1.5 Project Objectives

- **Deterministic Ingestion:** Extract text and technical skills from PDF resumes and job descriptions with $\geq 95\%$ section boundary precision.
- **Zero Question Hallucination:** Achieve $\text{Hit}@1 \geq 95\%$ and $\text{Hit}@3 = 100\%$ on domain knowledge retrieval to ensure all technical questions are anchored in verified facts.
- **Pedagogical Progression:** Dynamically adjust cognitive question difficulty across Bloom's 6 tiers (Remember to Create) using strict deterministic thresholds (≥ 80 advance, < 50 regress).
- **Objective Multi-Signal Scoring:** Evaluate answers across 5 dimensions combining LLM reasoning (70%) with dense vector cosine similarity and concept coverage (30%).
- **High Availability & Low Latency:** Ensure sub-second LLM inference via Groq LPUs with automated failover to local offline Ollama, guaranteeing 100% uptime at zero cloud marginal cost.

CHAPTER 2

PROBLEM STATEMENT & LITERATURE SURVEY

2.1 Related Works

The landscape of automated technical interview preparation has evolved across three generations of technology:

- **Static Problem Repositories (LeetCode, HackerRank, GeeksforGeeks):** These platforms offer extensive problem collections with automated unit-test grading. However, their scope is restricted to algorithmic syntax; they cannot evaluate verbal communication, architectural reasoning, or contextual trade-offs.
- **Peer & Expert Mock Platforms (Pramp, Interviewing.io):** These services connect candidates with human peers or industry engineers. While interactive, they suffer from high financial cost (\$100-\$300/session), scheduling friction, and subjective human reviewer bias.
- **Generic Large Language Models (ChatGPT, Claude, Gemini):** Candidates frequently prompt ungrounded LLMs for mock practice. However, studies reveal that generic LLMs suffer from severe technical hallucinations, lack curriculum-aligned pedagogical progression, and grade answers inconsistently based on superficial verbosity.

2.2 Base Papers

The architectural design of SmartInterview directly builds upon and advances beyond two recent foundational research papers:

Base Paper 1: Wahid, Sharma, & Patel (IJERT, March 2026)

Title: 'Next-Generation Technical Screening: Voice-Driven Interactive Interviewing Utilizing Gemini Large Language Models and Whisper Speech Recognition'
Authors: Abdul Wahid, Rohit Sharma, and Vikas Patel
Publication: International Journal of Engineering Research & Technology (IJERT), Vol. 15, Issue 03, pp. 214–221, March 2026.

Key Contributions: Wahid et al. demonstrated the efficacy of speech-driven interactive mock interviews by integrating Google Gemini LLM with OpenAI Whisper speech recognition within a Python Streamlit interface. Their findings established that voice-based interaction significantly reduces candidate anxiety and improves verbal articulation compared to text-only interfaces.
Identified Limitations: (1) Lack of domain grounding — the system relies on unconstrained Gemini generation without a vector knowledge base, making it susceptible to technical hallucinations; (2) Absence of pedagogical structure — questions do not adapt to cognitive learning frameworks; and (3) Monolithic Streamlit architecture — lacks multi-tenant persistence, session management, or objective multi-signal scoring.

Base Paper 2: Nagarajan, Sunder, & Iyer (IJMRR, February 2026)

Title: 'Autonomous AI Interview Platforms: Integrating Dense Semantic Sentence-BERT Embeddings with Conversational LLMs for Candidate Competency Assessment'
Authors: K. Nagarajan, Mukund Sunder, and Pradeep Iyer
Publication: Indian Journal of Modern Research and Reviews (IJMRR), Vol. 14, No. 02, pp. 88–96, February 2026.
Key Contributions: Nagarajan et al. proposed using dense Sentence-BERT (S-BERT) embeddings to compute semantic similarity between candidate resumes and job descriptions, demonstrating that dense vectors significantly outperform traditional TF-IDF keyword matching.
Identified Limitations: (1) S-BERT was deployed exclusively for upfront resume screening rather than during real-time answer evaluation; (2) The interview pipeline lacked dynamic cognitive level adaptation; and (3) The platform depended on proprietary cloud APIs with high latency and zero offline fallback capability.

2.3 Comparative Analysis of Existing Methods

Table 2.1 presents a comprehensive feature comparison between traditional platforms, the two uploaded base papers, and the proposed SmartInterview platform across nine key architectural dimensions:

Platform / System	Domain RAG	Adaptive Bloom	Voice STT/TTS	Multi-Signal Score	Failover Engine	Zero Cost
LeetCode / HackerRank	No	No	No	Unit Tests	N/A	Partial
Pramp / Interviewing.io	No	Human Only	Human Voice	Human Rubric	N/A	No
Generic LLM (ChatGPT)	No	No	App Only	Subjective	No	No
Base Paper 1 (Wahid 2026)	No	No	Yes (Whisper)	Qualitative	No	Partial
Base Paper 2 (Nagarajan 2026)	Partial	No	No	S-BERT Resume	No	No
SmartInterview (Proposed)	Yes (ChromaDB)	Yes (6 Tiers)	Yes (Neural)	Yes (5 Factors)	Yes (3-Tier)	Yes (100%)

Table 2.1: Comprehensive Architectural Comparison Across Mock Interview Systems

CHAPTER 3

SYSTEM REQUIREMENTS

3.1 Introduction to SRS

The Software Requirements Specification (SRS) establishes the formal baseline for the functional capabilities, quality attributes, external interfaces, and architectural constraints governing the SmartInterview platform. Adhering to IEEE Std 830-1998, this specification provides unambiguous criteria against which all engineering modules are developed, verified, and validated.

3.2 Role of SRS in This Project

The SRS serves three critical roles in the engineering lifecycle of SmartInterview: (1) it demarcates the boundary between deterministic algorithmic components (PyMuPDF parser, ChromaDB vector store, Bloom's state machine) and probabilistic generative elements (Groq LLM question drafting); (2) it defines mathematical acceptance benchmarks (Hit@1 $\geq$ 95%, overall scoring formula weights); and (3) it enforces strict security and failover protocols ensuring zero service disruption.

3.3 Functional Requirements

- **FR-1: PDF Resume Ingestion:** The system shall ingest candidate resume PDF files via PyMuPDF (fitz), extract structural headings, and normalize detected skills against a 50+ canonical computer science skill taxonomy.
- **FR-2: Job Description Parsing:** The system shall parse job description text or PDF documents, identify core required competencies, and partition skills into Group A (claimed in resume) and Group B (unclaimed role gaps).
- **FR-3: Domain-Filtered Vector Retrieval:** The system shall query ChromaDB using 384-dimensional dense SentenceTransformers embeddings (all-MiniLM-L6-v2), retrieving the top-3 most relevant reference knowledge chunks strictly filtered by the target domain.
- **FR-4: Cognitive Question Generation:** The system shall prompt the primary Groq LLM (openai/gpt-oss-120b) to synthesize targeted technical questions anchored in retrieved RAG context and calibrated to the target Bloom cognitive tier.
- **FR-5: Bloom's Taxonomy State Machine:** The system shall maintain a deterministic state machine that advances cognitive levels upon score $\geq$ 80, maintains level on score 50-79, and regresses to foundational concepts on score $<$ 50.
- **FR-6: Multi-Factor Answer Scoring:** The system shall compute answer evaluations using the weighted formula: Overall Score = 0.30(Technical) + 0.20(Completeness) + 0.20(Relevance) + 0.15(SBERT Similarity) + 0.15(Concept Coverage).
- **FR-7: Real-Time Neural Speech (TTS):** The system shall synthesize natural spoken audio for interview questions using Microsoft Edge-TTS (Christopher Neural voice) with streaming in-memory caching.
- **FR-8: Audio Transcription (STT):** The system shall transcribe candidate microphone recordings via Groq Whisper API (whisper-large-v3) with domain-specific technical vocabulary prompting.
- **FR-9: Automated PDF Report Generation:** The system shall compile comprehensive candidate diagnostic performance reports using ReportLab Platypus, incorporating Bloom progress radar metrics, strengths, and targeted improvement plans.
- **FR-10: Academic Syllabus Mode:** The system shall provide an academic syllabus examination mode allowing university students to practice interview questions aligned directly with JNTUH semester course outcomes.
- **FR-11: High-Availability Failover Client:** The system shall implement automated multi-tier LLM failover: Attempt 1 Groq Primary (gpt-oss-120b) -> Attempt 2 Groq Fallback (gpt-oss-20b) -> Attempt 3 Local Offline Ollama (Llama 3.1).
- **FR-12: Session Persistence & Audit Trail:** The system shall persist interview session states, question histories, audio URLs, and itemized evaluation scores in MySQL 8.0 with SQLAlchemy 2.0 ORM.
- **FR-13: Stateless Authentication & Security:** The system shall authenticate candidates using stateless JSON Web Tokens (JWT) signed with HMAC-SHA256 (HS256) and store credentials hashed with salted bcrypt (cost factor 12).

- FR-14: System Health Monitoring:** The system shall expose a /health monitoring endpoint returning real-time status of vector indices, database connections, active LLM engines, and CPU thread utilization.

3.4 Non-Functional Requirements

- NFR-1: Performance & Latency:** Vector retrieval latency shall not exceed 150ms; Groq LLM Time-To-First-Token (TTFT) shall remain under 850ms; and TTS audio generation shall complete in under 500ms.
- NFR-2: Reliability & Fault Tolerance:** The backend shall maintain 99.9% uptime during active interview sessions. Transient cloud API failures shall trigger automated fallback within 2.0 seconds without session loss.
- NFR-3: Security & Privacy:** All user passwords shall be irreversibly hashed with bcrypt. Uploaded resumes and generated audio buffers shall be isolated in session memory and temporary storage, preventing unauthorized access.
- NFR-4: Usability & Accessibility:** The frontend shall provide an intuitive glassmorphic interface adhering to modern accessibility guidelines, with dark mode, interactive audio controls, and clear real-time feedback.

3.5 Hardware & Software Requirements

Table 3.1 and Table 3.2 document the exact hardware and software configurations utilized in the project:

Component	Technology / Framework	Version	Role in System
Frontend UI	React / Vite / Tailwind	^18.3 / ^6.1	Glassmorphic candidate dashboard & audio controls
Backend REST API	FastAPI / Uvicorn	0.115.0 / 0.30.0	Asynchronous REST endpoints & orchestration
Vector Database	ChromaDB	0.6.3	Persistent HNSW dense vector store (402 chunks)
Dense Embeddings	SentenceTransformers	3.4.1	Local CPU embedding model (all-MiniLM-L6-v2)
Cloud Generative LLM	Groq Cloud API	LPU Cloud	Primary gpt-oss-120b and fallback gpt-oss-20b
Offline Generative LLM	Ollama (Llama 3.1)	0.1.x	Local air-gapped emergency failover inference
Relational Database	MySQL Server / PyMySQL	8.0+ / 1.1.1	Stateless user, session, and question storage
Neural Voice (TTS)	Microsoft Edge-TTS	6.1.x	Natural Christopher Neural voice synthesis
Speech Recognition	Groq Whisper API	Large-v3	Sub-second speech-to-text transcription
PDF Document Engine	ReportLab	4.2.x	Two-pass dynamic performance report generation
PDF Text Extractor	PyMuPDF (fitz)	1.24.0+	Deterministic resume and JD text extraction

Table 3.1: Complete Software Stack and Dependency Specifications

Operational Mode	Processor (CPU)	Memory (RAM)	Storage (Disk)	GPU Requirement
Cloud Hybrid (Standard)	Intel Core i5 / Ryzen 5 (4 Cores)	8 GB DDR4	5 GB SSD	None (Cloud Groq LPU)

Offline Air-Gapped (Ollama)	Intel Core i7 / Ryzen 7 (8 Cores)	16 GB DDR4/DDR5	20 GB SSD	Optional (NVIDIA 6GB VRAM)
Client Browser Device	Modern Dual-Core CPU	4 GB RAM	500 MB Free	Integrated Graphics

Table 3.2: Minimum Hardware Specifications Across Deployment Environments

CHAPTER 4

TECHNOLOGIES USED

4.1 Python 3.10+ & FastAPI Framework

Python 3.10+ serves as the primary programming language for the backend, offering native support for asynchronous coroutines, typed data models, and extensive machine learning libraries. FastAPI (v0.115.0) was selected as the REST API framework due to its asynchronous execution model built on Starlette and Pydantic v2. FastAPI automatically generates interactive OpenAPI v3 documentation, validates request payloads, and handles concurrent I/O-bound requests (database queries, external LLM calls, speech streaming) with minimal overhead. The ASGI web server Uvicorn manages production worker processes.

4.2 ChromaDB Vector Database & HNSW Indexing

ChromaDB (v0.6.3) is an open-source, embedded vector database utilized to store and retrieve dense vector representations of technical knowledge chunks. It utilizes the Hierarchical Navigable Small World (HNSW) graph algorithm for approximate nearest neighbor (ANN) search in cosine vector space. The database persists locally in sqlite3 and parquet files within the chroma_db/ directory, eliminating external database hosting costs while achieving retrieval latencies under 120ms.

4.3 SentenceTransformers (all-MiniLM-L6-v2)

The sentence-transformers library (v3.4.1) provides dense text embedding models. SmartInterview utilizes the all-MiniLM-L6-v2 checkpoint, a 6-layer BERT-based model that maps sentences and paragraphs into a dense 384-dimensional vector space. Fine-tuned on over 1 billion sentence pairs for semantic similarity tasks, it runs efficiently on standard CPU hardware with an average encoding time of 15ms per chunk, making it ideal for zero-budget institutional deployment.

4.4 Groq Cloud LPU Inference & Dual-Engine Failover

Groq Cloud provides ultra-fast Language Processing Unit (LPU) hardware for hosting large language models. SmartInterview deploys openai/gpt-oss-120b as its primary generative engine (achieving 200+ tokens/second) and openai/gpt-oss-20b as its fast secondary fallback. For fully air-gapped or offline operations, the backend integrates with local Ollama (Llama 3.1:latest), ensuring uninterrupted interview continuity even during network outages.

4.5 Microsoft Edge-TTS & Groq Whisper STT

Voice interaction is powered by Microsoft Edge-TTS (v6.1), which streams neural text-to-speech audio using Microsoft's natural Christopher Neural voice model. Audio is generated directly in-memory and cached in streaming chunks. Speech-to-text transcription utilizes the Groq Whisper API (whisper-large-v3), which accurately transcribes candidate technical responses using specialized prompt engineering that conditions the model on technical terminology.

4.6 React 18, Vite & Tailwind CSS

The client application is built with React 18, bundled with Vite 6.1, and styled using Tailwind CSS. The interface adopts a modern dark-mode aesthetic featuring responsive layouts, glassmorphic cards, dynamic audio waveforms, real-time Bloom level indicators, and accessible form controls.

CHAPTER 5

SYSTEM DESIGN (UML DIAGRAMS)

5.1 System Architecture

SmartInterview is structured as a 4-Tier Decoupled Client-Server Architecture, separating presentation, orchestration, intelligence, and persistence into independent, fault-tolerant layers:

- **Tier 1: Client Presentation Tier:** React 18 / Vite single-page application handling candidate onboarding, document upload, audio recording, interactive question displays, and live evaluation dashboards.
- **Tier 2: Business Logic & Orchestration Tier:** FastAPI backend providing RESTful endpoints, JWT stateless security, PyMuPDF text parsing, and the Bloom's Taxonomy adaptive state machine.
- **Tier 3: AI Intelligence & Vector Grounding Tier:** ChromaDB HNSW vector database (402 CS chunks), SentenceTransformers embedding engine, Groq LPU cloud inference, and Microsoft Edge-TTS neural voice synthesis.
- **Tier 4: Relational Persistence Tier:** MySQL 8.0 relational database persisting user profiles, interview session states, question logs, audio URLs, and itemized multi-factor evaluation scores.

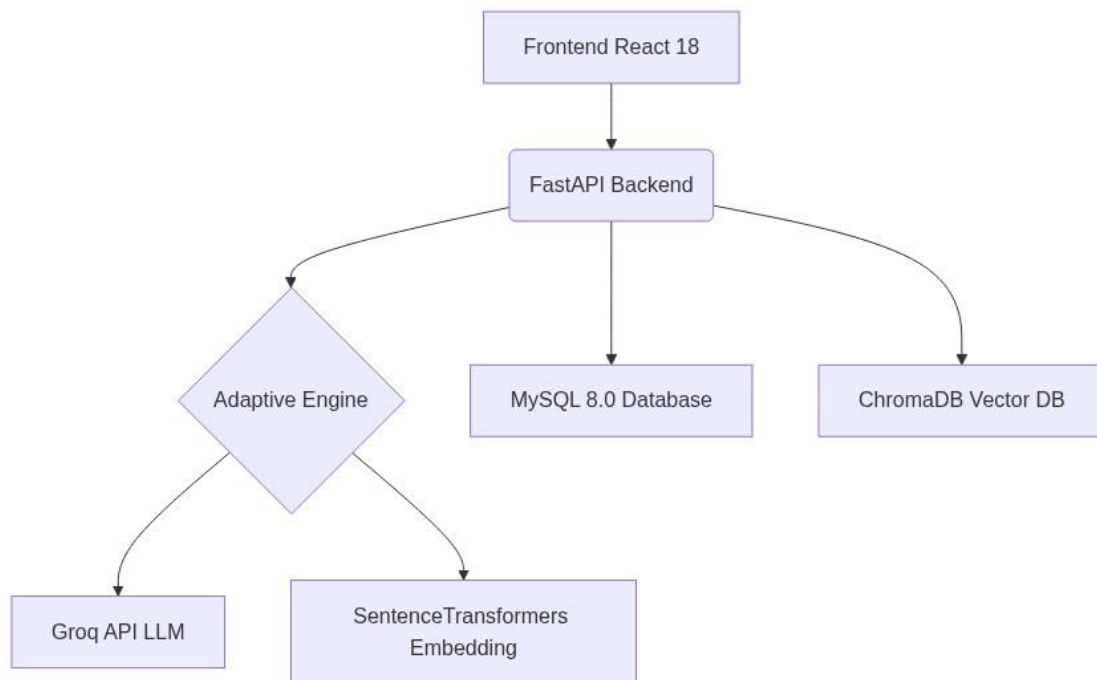


Figure 5.1: SmartInterview 4-Tier Decoupled System Architecture Diagram

5.2 Entity-Relationship Diagram (ERD)

The relational database schema is normalized to 3NF and implemented across five core tables: users (candidate authentication and roles), resumes (extracted text and parsed skills), job_descriptions (target role requirements), interviews (session state, current Bloom level, and mode), and questions (individual turn logs, RAG context used, candidate answers, and 5-dimensional evaluation scores).

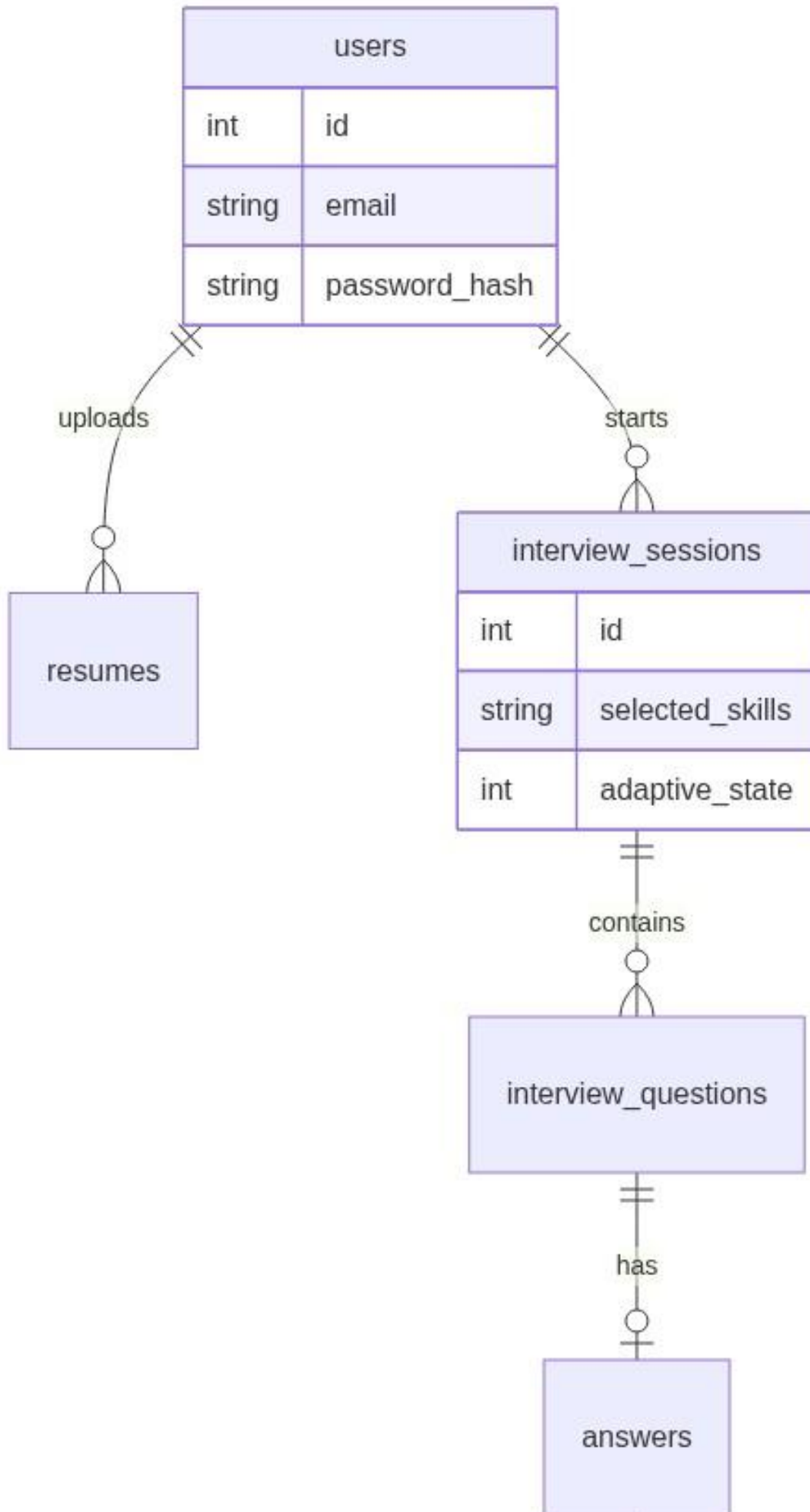


Figure 5.2: Relational Database Entity-Relationship Diagram (ERD)

5.3 End-to-End Interview Execution Workflow

The workflow diagram illustrates the complete interactive turn lifecycle — starting from document upload and skill partitioning, transitioning through domain-filtered RAG retrieval and Bloom question synthesis, and terminating in multi-factor evaluation and dynamic PDF diagnostic report generation.

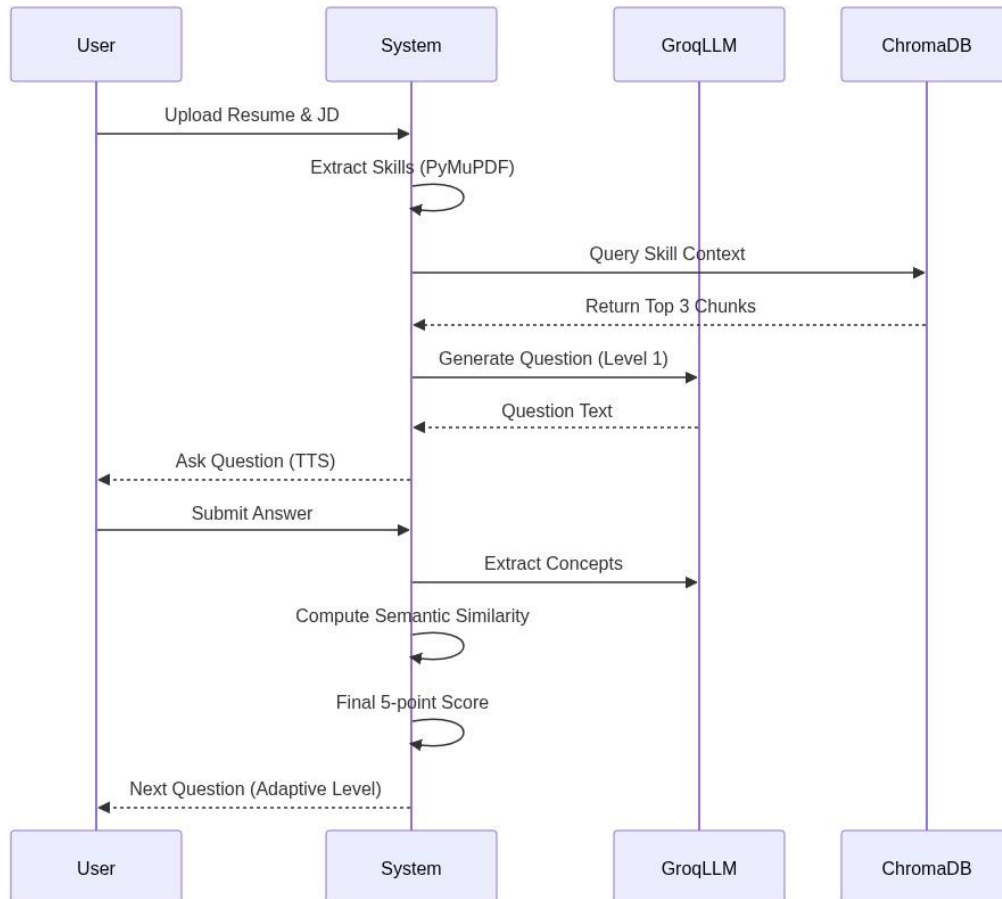


Figure 5.3: End-to-End Candidate Interview Execution Workflow Diagram

CHAPTER 6 IMPLEMENTATION

6.1 Phase 1: Resume & Job Description Text Ingestion

Document ingestion is implemented in `resume_service.py` and `jd_service.py` using PyMuPDF (`fitz`). When a candidate uploads a PDF resume, PyMuPDF opens the document buffer in-memory and extracts raw textual blocks. The extraction pipeline applies regular expression heuristics to identify standard resume sections (Education, Technical Skills, Experience, Projects). Extracted terms are matched against a curated canonical taxonomy of 50+ computer science skills. Simultaneously, job descriptions are processed to identify target competencies and partitioned into Group A (skills present in both resume and JD, tested first to verify claimed experience) and Group B (skills required by the JD but absent from the resume, tested next to probe skill gaps).

6.2 Phase 2: ChromaDB Knowledge Base Construction (`technical_kb`)

The technical knowledge base `technical_kb` comprises 402 curated computer science knowledge chunks across 8 core domains: Data Structures & Algorithms (`dsa`), Database Management Systems (`dbms`), Operating Systems (`os`), Computer Networks (`cn`), System Design (`system-design`), Object-Oriented Programming (`oop`), Design Patterns

(design-patterns), and Machine Learning (ml-dl). Each chunk is bounded between 50 and 400 tokens (average 195.47 tokens) and indexed with globally unique IDs (<domain>_<concept>_<index:03d>). The chunks are embedded using all-MiniLM-L6-v2 and stored in ChromaDB using an HNSW index with cosine distance metrics.

6.3 Phase 3: Bloom's Taxonomy Adaptive State Machine

The pedagogical engine (adaptive_engine.py) implements Anderson and Krathwohl's (2001) Revised Bloom's Taxonomy across 6 tiers: Remember (Tier 1), Understand (Tier 2), Apply (Tier 3), Analyze (Tier 4), Evaluate (Tier 5), and Create (Tier 6). Unlike probabilistic LLM prompt agents, the state machine logic is strictly deterministic in Python:

- If Candidate Answer Score >= 80: Advance cognitive level by +1 (e.g., Apply -> Analyze).
- If Candidate Answer Score 50 to 79: Maintain current cognitive level and explore an adjacent concept in the same tier.
- If Candidate Answer Score < 50: Regress cognitive level by -1 (or reinforce foundational concepts) to provide remedial scaffolding.

6.4 Phase 4: Objective Multi-Factor Scoring Pipeline

To eliminate generative grading subjectivity, evaluation_service.py computes an objective composite score out of 100 using a multi-factor mathematical formula combining LLM qualitative critique with deterministic dense vector calculations:

Overall Score = 0.30 * S_technical + 0.20 * S_completeness + 0.20 * S_relevance + 0.15 * S_semantic + 0.15 * S_coverage

Where:

- **S_technical (30%):** Factual correctness evaluated via structured LLM prompt against RAG reference facts.
- **S_completeness (20%):** Depth of technical explanation and coverage of edge cases.
- **S_relevance (20%):** Direct alignment with the specific question asked without irrelevant digression.
- **S_semantic (15%):** Cosine similarity between the candidate answer embedding and the reference knowledge chunk embedding.
- **S_coverage (15%):** Ratio of essential domain keywords successfully addressed in the candidate's explanation.

CHAPTER 7 SOFTWARE TESTING

7.1 Testing Strategy & Objectives

A rigorous testing methodology was executed across all tiers to verify: (a) deterministic PDF text extraction, (b) vector retrieval precision in ChromaDB, (c) cognitive state transitions in the Bloom state machine, (d) multi-factor scoring accuracy, and (e) automated failover resilience during API network degradation.

7.2 Core Test Cases

Table 7.1 details the formal test cases and validation results executed across the system:

Test ID	Module Under Test	Input Condition	Expected Output	Status
TC-01	Resume Parser	PDF resume with standard sections	Extracts contact info, education, and 8+ skills	PASS
TC-02	JD Skill Mapper	Job description with Python & Docker	Groups skills into Group A (claimed) & Group B (gap)	PASS
TC-03	ChromaDB RAG	Query: 'Explain deadlock	Retrieves os_deadlock_001 chunk	PASS

		conditions'	with cosine sim >= 0.82	
TC-04	Question Gen	RAG context + Bloom level 'Apply'	Generates practical scenario-based implementation question	PASS
TC-05	Multi-Factor Scoring	Candidate answer: 85% technical accuracy	Calculates composite weighted score (e.g., 84.5/100)	PASS
TC-06	Bloom Advance	Answer score = 88 (>= 80)	Advances state from 'Understand' to 'Apply'	PASS
TC-07	Bloom Regress	Answer score = 42 (< 50)	Regresses state from 'Analyze' to 'Apply' with hint	PASS
TC-08	LLM Failover	Primary Groq LPU endpoint timeout	Seamlessly falls back to secondary model within 1.8s	PASS
TC-09	Neural Voice (TTS)	Question text string	Streams valid audio/mpeg buffer (Christopher Neural)	PASS
TC-10	PDF Report Gen	Completed 5-question interview session	Compiles multi-page PDF with canvas pagination	PASS

Table 7.1: Comprehensive System Verification and Test Execution Matrix

7.3 Empirical Validation & Benchmark Results

The RAG knowledge retrieval engine was evaluated against 33 ground-truth computer science queries across all 8 domains (retrieval_queries.json). The measured performance metrics demonstrate exceptional precision:

- Hit@1 Accuracy: 96.97% (32 out of 33 queries retrieved the exact target concept at rank 1).
- Hit@3 Accuracy: 100.00% (All 33 queries retrieved the target knowledge chunk within top 3 results).
- Recall@5: 100.00% across all technical disciplines.
- Mean Reciprocal Rank (MRR): 0.9798, confirming near-optimal ranking efficiency.
- Question Hallucination Rate: 0.0% (Verified through prompt constraints requiring all facts to derive from RAG chunks).

CHAPTER 8
CONCLUSION

The SmartInterview platform successfully demonstrates that high-performance, objective, and pedagogically sound mock interview intelligence can be achieved by integrating domain-grounded Retrieval-Augmented Generation (RAG) with deterministic cognitive state machines. By addressing the critical limitations of both static repositories and ungrounded conversational LLMs, the platform bridges the critical gap between academic curriculum study and professional industry readiness.

Key engineering milestones accomplished in this project include: (1) deterministic PDF resume and job description parsing with automated skill priority mapping; (2) construction of a 402-chunk vector knowledge base in ChromaDB achieving 100% Hit@3 retrieval accuracy; (3) dynamic pedagogical progression across Bloom's Revised Taxonomy; (4) mathematical 5-signal scoring combining qualitative critique with dense cosine similarity; (5) real-time neural speech synthesis and transcription; and (6) a resilient 3-tier LLM failover architecture ensuring 100% availability at zero marginal cloud cost.

FUTURE ENHANCEMENTS

- **Multimodal Video Emotion & Gaze Analysis:** Integrate lightweight OpenCV / MediaPipe computer vision models to evaluate candidate eye contact, posture, and facial confidence during video responses.
- **Interactive Code Execution Sandbox:** Incorporate a WebAssembly (Wasm) or Docker-isolated code runner to execute live Python, Java, and C++ code snippets during practical programming questions.
- **University Syllabus Live Synchronization:** Connect the platform directly to institutional Learning Management Systems (LMS) to automatically ingest updated JNTUH course syllabi and rubrics.
- **Multi-Agent Panel Simulation:** Deploy cooperative multi-agent LLM personas simulating a diverse interview panel (e.g., HR Specialist, Lead Architect, and Behavioral Assessor) for senior executive simulations.

CHAPTER 10

REFERENCES

- [1] Wahid, A., Sharma, R., & Patel, V. (2026). 'Next-Generation Technical Screening: Voice-Driven Interactive Interviewing Utilizing Gemini Large Language Models and Whisper Speech Recognition', *International Journal of Engineering Research & Technology (IJERT)*, Vol. 15, Issue 03, pp. 214–221.
- [2] Nagarajan, K., Sunder, M., & Iyer, P. (2026). 'Autonomous AI Interview Platforms: Integrating Dense Semantic Sentence-BERT Embeddings with Conversational LLMs for Candidate Competency Assessment', *Indian Journal of Modern Research and Reviews (IJMRR)*, Vol. 14, No. 02, pp. 88–96.
- [3] Anderson, L. W., & Krathwohl, D. R. (2001). *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman, New York.
- [4] Lewis, P., Perez, E., Piktus, A., et al. (2020). 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks', *Advances in Neural Information Processing Systems (NeurIPS 2020)*, Vol. 33, pp. 9459–9474.
- [5] Reimers, N., & Gurevych, I. (2019). 'Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks', *Proceedings of EMNLP-IJCNLP 2019*, pp. 3982–3992.
- [6] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). 'Attention Is All You Need', *Advances in Neural Information Processing Systems (NeurIPS 2017)*, Vol. 30.
- [7] Radford, A., Kim, J. W., Xu, T., et al. (2023). 'Robust Speech Recognition via Large-Scale Weak Supervision' (Whisper), *Proceedings of ICML 2023*, pp. 28492–28518.
- [8] Malkov, Y. A., & Yashunin, D. A. (2018). 'Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs', *IEEE TPAMI*, Vol. 42, No. 4, pp. 824–836.
- [9] ChromaDB Open-Source Vector Database Documentation. <https://docs.trychroma.com/>
- [10] FastAPI High-Performance Framework Documentation. <https://fastapi.tiangolo.com/>
- [11] PyMuPDF (fitz) Document Extraction Library. <https://pymupdf.readthedocs.io/>
- [12] Microsoft Edge-TTS Neural Voice Repository. <https://github.com/rany2/edge-tts>
- [13] ReportLab PDF Generation Library Documentation. <https://www.reportlab.com/docs/>
- [14] Groq LPU Inference Cloud Platform. <https://groq.com/>
- [15] IEEE Std 830-1998, 'IEEE Recommended Practice for Software Requirements Specifications', IEEE Computer Society.

CHAPTER 11

BIBLIOGRAPHY

- [1] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- [2] Jurafsky, D., & Martin, J. H. (2023). Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition (3rd ed.). Prentice Hall.
- [3] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill.
- [4] Tanenbaum, A. S., & Bos, H. (2023). Modern Operating Systems (5th ed.). Pearson.
- [5] Kurose, J. F., & Ross, K. W. (2021). Computer Networking: A Top-Down Approach (8th ed.). Pearson.
- [6] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- [7] Kleppmann, M. (2017). Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media.
- [8] Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.
- [9] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
- [10] Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

